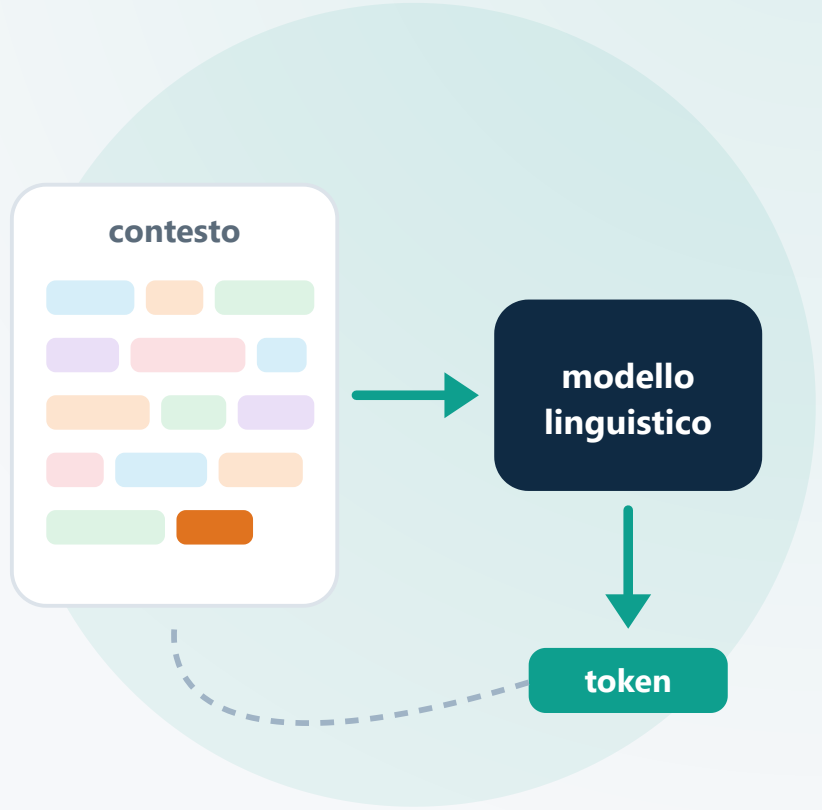
MODULO · FONDAMENTI DI IA 2

Come funziona un modello linguistico

Language model, LLM e SLM, token, contesto e prompt

Giacomo Bocchese



Ripasso: cos'è un modello linguistico

Un **modello linguistico** (in inglese *language model*) è un modello che riceve un testo e **prevede come continua**.



1. È una **rete neurale** di tipo transformer, con moltissimi parametri.
2. È stato addestrato su **enormi quantità di testo**: ha imparato a prevedere la parola che viene dopo.

Ripasso: il modello scrive una parola alla volta

1. Per ogni parola possibile il modello calcola una **probabilità**.
2. Ne **sceglie** una, la aggiunge al testo e ripete.
3. Ripetendo questo passo, parola dopo parola, scrive una **risposta intera**.



Per prevedere ogni parola il modello **rilegge tutto il testo**, dall'inizio, compresa la parola appena aggiunta.

Contesto, modello e risposta



↺ Ogni parola generata viene aggiunta al contesto. Per prevedere la successiva il modello rilegge tutto il contesto.

1. Il **contesto** è tutto il testo che il modello legge prima di rispondere.
2. Il **prompt**, cioè ciò che scriviamo noi, è solo una parte del contesto.

Large language model (LLM)

1. Un **LLM** è un modello linguistico molto grande: ha **più di 50 miliardi** di parametri, spesso centinaia o migliaia di miliardi.
2. Funziona in grandi **centri di calcolo** (data center): lo usiamo attraverso internet ed è impossibile eseguirlo su un normale computer.
3. È **più intelligente**: scrive bene in molte lingue e svolge compiti complessi.

GPT-6
OpenAI · ChatGPT

Claude Opus 5.5
Anthropic

Gemini 4
Google

DeepSeek V4
DeepSeek

Qwen 3.8 Max
Alibaba

Kimi K3
Moonshot AI

OpenAI, Anthropic e Google non dichiarano quanti parametri hanno i loro modelli. I più grandi modelli pubblici superano i **mille miliardi**.

Small language model (SLM)

1. Uno **SLM** è un modello linguistico abbastanza piccolo da funzionare su un normale **computer** o su un **telefono**.
2. Ha **meno di 20 miliardi** di parametri, spesso solo pochi miliardi.
3. Funziona anche senza internet e i dati restano sul dispositivo, ma è **meno intelligente** e sbaglia più spesso.

LFM3 1.7B
Liquid AI · 1,7 miliardi · ott 2026 · quello della prova

Qwen 3.5 Small
Alibaba · da 0,8 a 9 miliardi · mar 2026

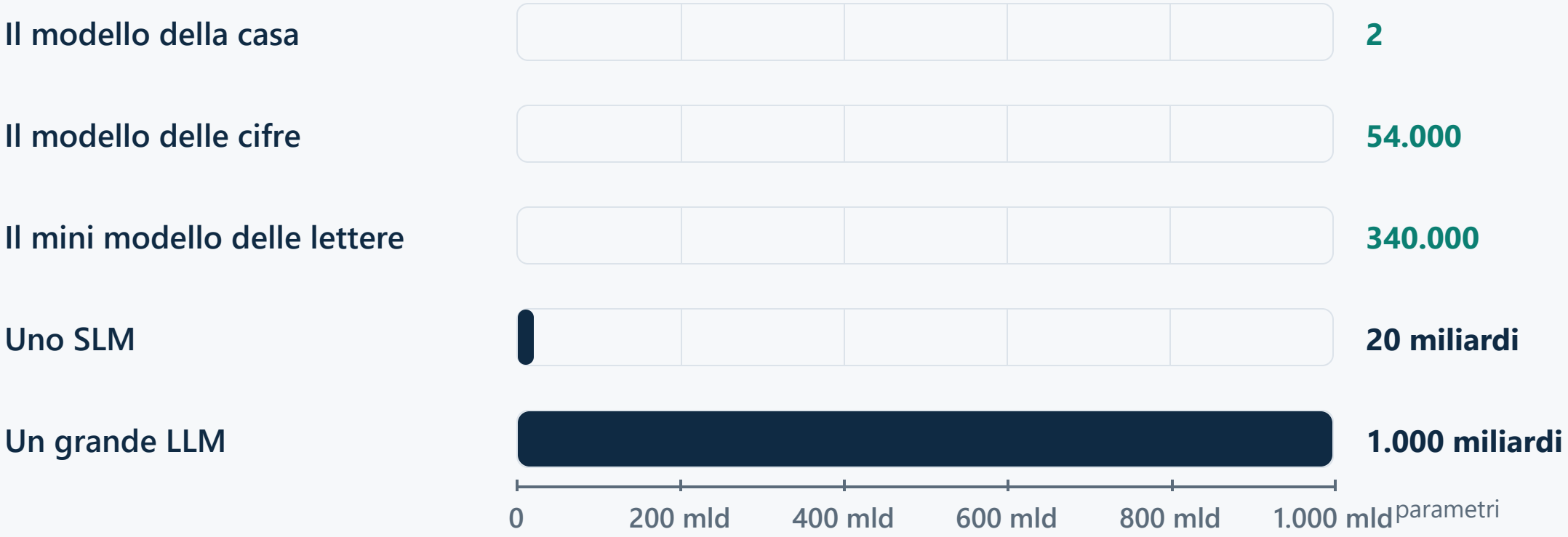
Gemma 4 E2B / E4B
Google · circa 2 e 4 miliardi · apr 2026

Ministral 3
Mistral · 3, 8 e 14 miliardi · dic 2025

Gemini Nano
Google · nei telefoni Android

Quanti parametri: dal modello della casa agli LLM

Scala lineare | Scala logaritmica



Come nasce un assistente: le fasi dell'addestramento

1

Pre-addestramento

Il modello legge enormi quantità di testo (siti, libri, codice) e impara a prevedere il token successivo. Ne esce un **modello di base**.

→

2

Addestramento con esempi

Persone scrivono migliaia di domande con la risposta ideale: il modello impara a rispondere come un assistente.

→

3

Addestramento al ragionamento

Il modello risolve problemi di matematica e programmazione con una soluzione verificabile e viene premiato quando arriva al risultato giusto.

→

4

Rinforzo con il giudizio umano

(*RLHF*) Il modello scrive più risposte alla stessa domanda, delle persone indicano la migliore e il modello si regola per dare risposte simili.

Le **conoscenze** vengono quasi tutte dalla fase 1. Il **modo di rispondere e di ragionare** viene dalle fasi 2, 3 e 4.

Lo stesso testo, due modelli diversi

TESTO DI INGRESSO Qual è la capitale della Francia?

Modello di base

solo fase 1

Qual è la capitale della Francia? **Qual è la capitale della Germania? Qual è la capitale della Spagna? Qual è la capitale…**

Ha visto molti elenchi di domande: continua il testo scrivendone altre.

Assistente

fasi 1, 2, 3 e 4

La capitale della Francia è Parigi.

Ha imparato che a una domanda segue una risposta.

Il modello di base non sbaglia: fa quello per cui è addestrato, cioè **continuare il testo**.

Il modello e l'app non sono la stessa cosa

1. ChatGPT, Claude e Gemini sono **app**. GPT, Claude Opus e Gemini Pro sono i **modelli** che ci girano dentro.
2. Lo stesso modello può comportarsi in modo diverso in app diverse, perché cambiano il **prompt di sistema** e gli **strumenti**.
3. Un programma può usare il modello direttamente, via **API**, senza l'app: non ci sono prompt di sistema né strumenti, a meno che non li aggiunga chi scrive il programma.



Come leggere i nomi dei modelli

NEL NOME	COSA INDICA DI SOLITO	QUANDO USARLO
`mini` `nano` `flash` `haiku` `lite`	Modello più **piccolo**: veloce ed economico, meno capace	Compiti semplici e ripetuti
`pro` `opus` `fable` `astra`	Modello più **grande**: più capace, più lento e costoso	Compiti difficili
`thinking` `reasoning` `deep think`	Modello di **ragionamento**	Matematica, logica, codice
`5` `5.5` `6`	**Generazione**: più alto vuol dire più recente	

Nelle app il modello si sceglie da un menu. Per un'email basta un modello veloce; per un problema difficile serve un modello **grande** o di **ragionamento**.

PRATICA · IN COPPIA · ARENA E LIQUID PLAYGROUND

Grande contro piccolo

1 Il modello grande

Apri **arena.ai**, scegli la modalità **Direct** e un modello grande, per esempio **claude-sonnet-5.5**.

2 Il modello piccolo

Apri **playground.liquid.ai** e scegli **LFM3-1.7B**: un modello da 1,7 miliardi di parametri, uscito a ottobre 2026.

3 Fai le stesse domande

«Riassumi la seconda guerra mondiale in tre righe.»

«Quante lettere r ci sono in «il ramarro marrone corre»?»

«Quante lettere "r" ci sono in «il ramarro marrone corre»?»

4 Un problema di ragionamento

«Ho 3 mele, ne mangio una, ne compro altre 5 e taglio a metà tutte le mele che ho. Due metà marciscono. Quante mezze mele posso ancora mangiare?»

→ Confronta correttezza, chiarezza e velocità.

PRATICA · IN COPPIA · ARENA, LIQUID PLAYGROUND E ONECOMPILER

Grande contro piccolo: scrivere una pagina web

1 Scegli una sfida

«Crea un unico file HTML con un gioco: un quadrato compare in punti casuali e bisogna cliccarlo. Mostra i punti e un timer di 30 secondi.»

«Crea un unico file HTML con un orologio analogico che segna l'ora attuale.»

2 Oppure un disegno

«Crea un unico file HTML con un disegno in SVG: un paesaggio con montagne, un lago, il sole e una casetta.»

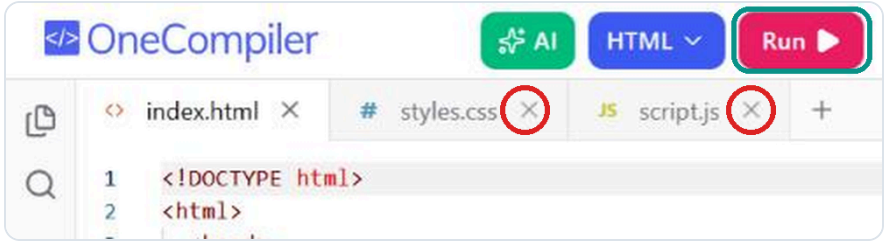
3 Chiedi a entrambi i modelli

Invia la stessa sfida al modello grande su **arena.ai** e al modello piccolo su **playground.liquid.ai**. Copia il codice che scrivono.

→ La pagina funziona? Quale è fatta meglio?

4 Guarda il risultato

Apri **onecompiler.com/html** e chiudi con la × i file **styles.css** e **script.js**. In **index.html** cancella tutto, incolla il codice e premi **Run**.



Hai notato cosa fa il modello quando sbaglia?

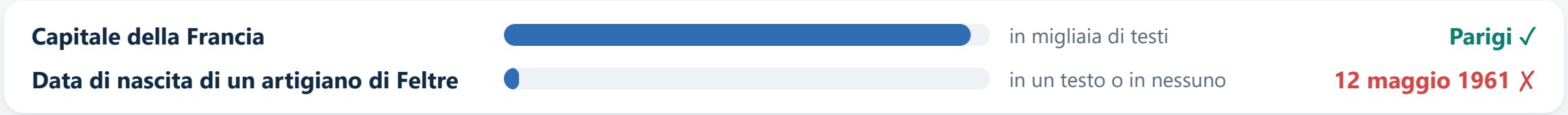
1. Non dice «non lo so»: **risponde comunque**, con lo stesso tono sicuro di quando ha ragione.

2. Per questo tocca a chi lo usa **verificare** le risposte e **provare** il codice prima di fidarsi.

Una risposta che contiene informazioni **false o inventate**, presentate come se fossero vere, si chiama **allucinazione** (in inglese *hallucination*).

Perché il modello inventa

1. **Scrive testo probabile, non testo verificato.** Il modello sceglie le parole più probabili dopo quelle precedenti. Non ha un passaggio in cui controlla se un fatto è vero: una frase falsa ma plausibile può avere una probabilità alta.*
2. **I fatti rari non si possono ricavare.** La capitale della Francia compare in migliaia di testi e il modello la conosce. La data di nascita di una persona poco nota compare una volta o mai: non c'è una regola da cui ricavarla, quindi il modello scrive una data plausibile, spesso sbagliata.



* I modelli agentici più avanzati (per esempio Claude Opus 5.5 e GPT-6.1-sol) possono verificare i fatti durante il ragionamento, cercando sul web, se il prompt chiede di farlo. Non è comunque una garanzia che la risposta sia corretta.

> Più l'informazione è **rara e precisa** (nomi, date, numeri, citazioni, articoli di legge), più va **verificata**.

Il modello legge e scrive token, non parole

`Il` `gat` `to` `è` `sal` `ito` `sul` `tet` `to` `.`	**10** token
`The` `cat` `climbed` `onto` `the` `roof` `.`	**7** token

1. Un **token** è un pezzo di testo: una parola intera, una parte di parola o un segno di punteggiatura.
2. Finora abbiamo detto «parola» per semplicità: in realtà il modello prevede il **token successivo**.
3. La stessa frase in italiano richiede di solito **più token** che in inglese.

Quanto è lungo un testo in token

TOKEN	IN ITALIANO, CIRCA
1 token	⅔ di parola
1.000 token	650 parole: una pagina e mezza
100.000 token	65.000 parole: un romanzo di 250 pagine
1.000.000 token	650.000 parole: circa tre volte *I promessi sposi*

Cosa dipende dal numero di token

1. Il **costo** di utilizzo del modello: chi lo usa paga in base ai token usati.
2. La **quantità massima di testo** che il modello può elaborare insieme: la finestra di contesto.
3. I **limiti di utilizzo** delle app: dopo un certo numero di token l'app chiede di aspettare.

MODELLO	PREZZO PER MILIONE DI TOKEN	TESTO MASSIMO (TOKEN)
Claude Opus 5.5 Anthropic	12 $	1 milione
GPT-6 Sol OpenAI	6 $	1,1 milioni
DeepSeek V4 Pro DeepSeek	2,6 $	1 milione
Claude Haiku 5.5 Anthropic	0,30 $	1 milione
Gemma 4 31B Google	0,30 $	128.000
LFM3 1.7B Liquid AI	0,14 $	256.000

PRATICA · IN COPPIA · PLATFORM.OPENAI.COM/TOKENIZER

Proviamo a dividere un testo in token

1 Apri il tokenizer

Vai su **platform.openai.com/tokenizer**: non serve un account. Incolla un testo e guarda i pezzi colorati.

2 Italiano e inglese

«Buongiorno, le scrivo per chiedere informazioni sulla fattura.»

«Good morning, I am writing to ask for information about the invoice.»

3 Una parola lunga

«precipitevolissimevolmente»

→ In quanti pezzi viene divisa?

4 Un tuo testo

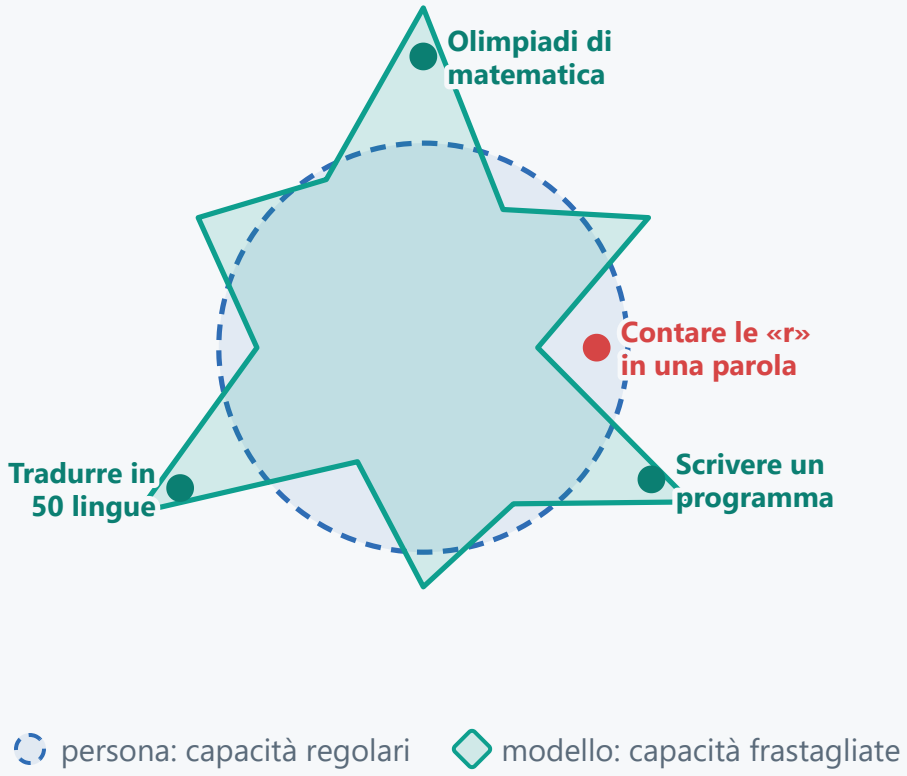
Incolla un'email o un paragrafo che usi al lavoro.

→ Quanti token sono? E quante parole?

Jagged intelligence: capacità irregolari

1. I modelli di metà 2025 hanno raggiunto il livello da **medaglia d'oro** alle Olimpiadi internazionali di matematica, ma possono sbagliare compiti che un bambino fa senza fatica, come **contare le «r»** in una parola.

2. Non si può dedurre «è bravo in X, quindi è bravo anche in Y», come si fa con una persona: **una persona brava in X di solito è brava anche in Y**, un modello no.

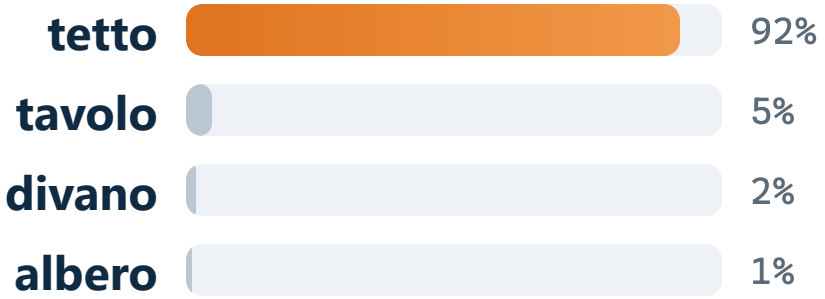


La temperatura regola quanto è varia la scelta

Token successivo dopo «Il gatto è salito sul…»

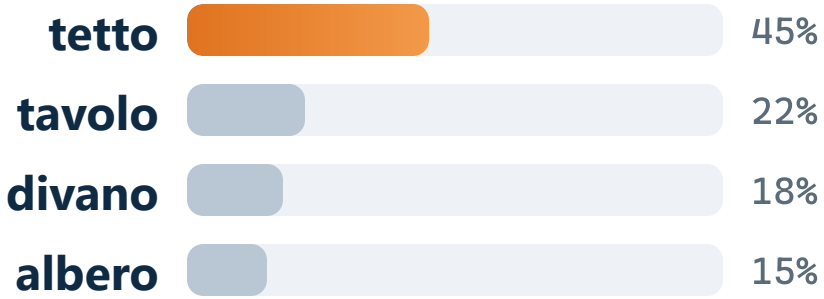
Temperatura bassa

Quasi sempre il token più probabile: risposte prevedibili e ripetibili



Temperatura alta

Più spazio ai token meno probabili: risposte varie, ma più rischiose

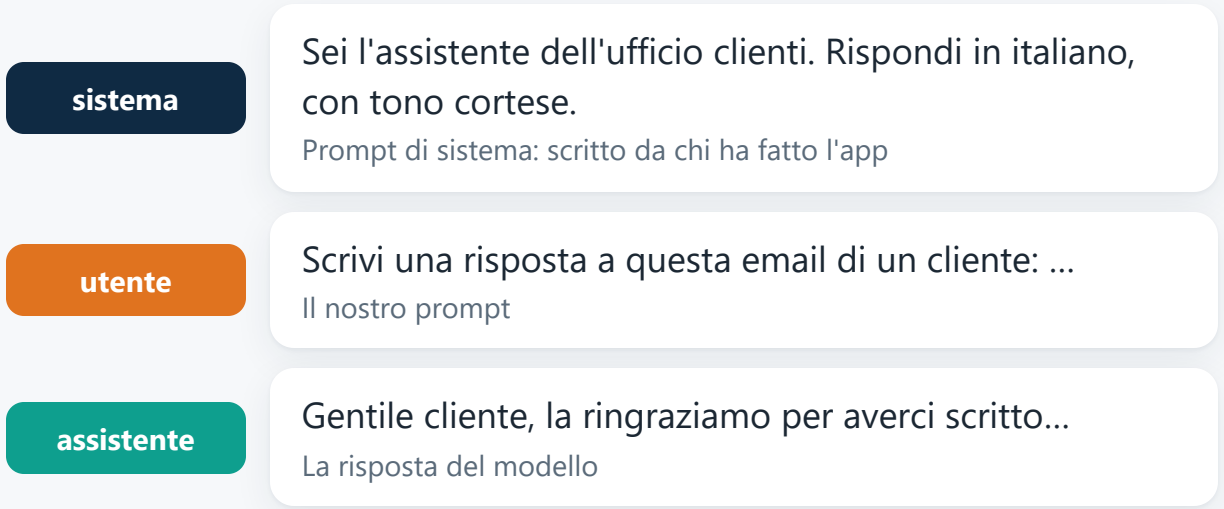


Nelle app di solito non si regola. Nella simulazione delle lettere è il cursore **«Creatività (temperatura)»**.

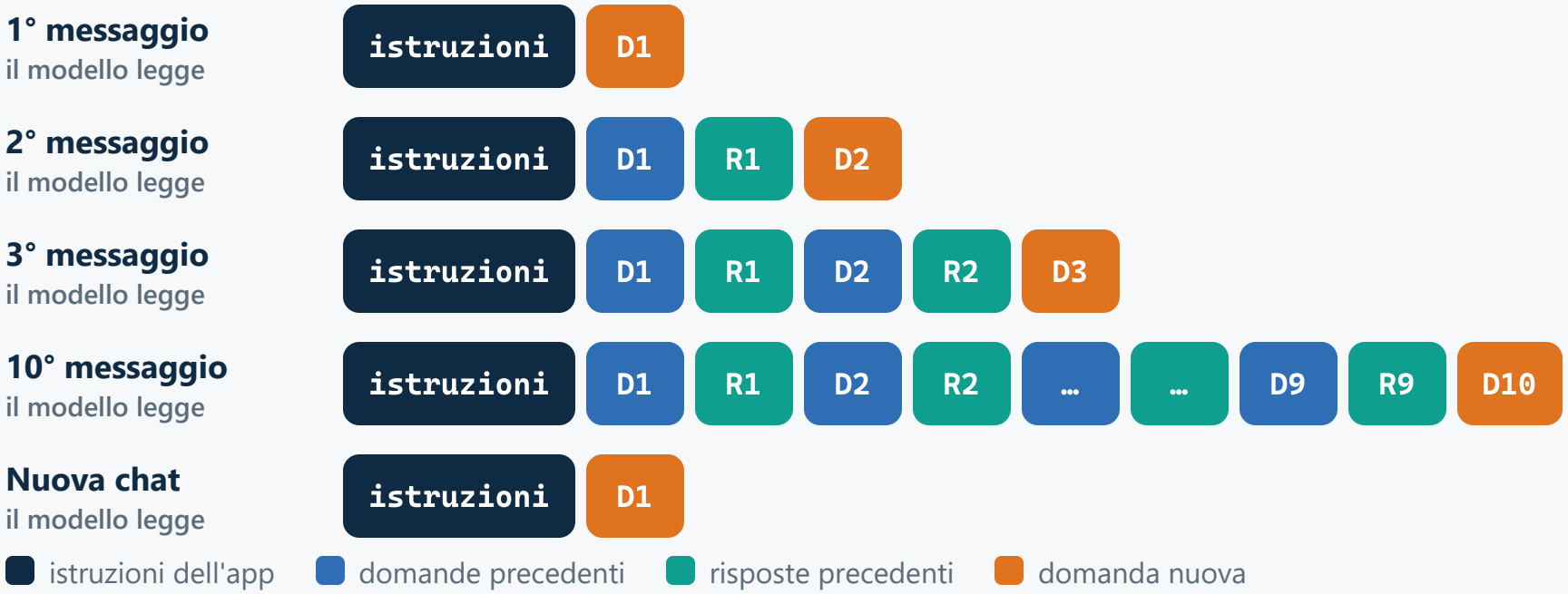
Il prompt è la richiesta che scriviamo al modello

Il **prompt** è il testo che scriviamo per ottenere una risposta: una domanda, un'istruzione o un testo da elaborare.

1. Il prompt è la parte del contesto che **scriviamo noi**, a ogni messaggio.
2. Il **prompt di sistema** (*system prompt*) è scritto da chi ha costruito l'app: il modello lo legge prima della chat e di solito noi non lo vediamo.



A ogni messaggio il modello rilegge tutta la chat



Il modello non "ricorda" la chat: **la rilegge ogni volta**. Una nuova chat parte da un contesto vuoto.

Il contesto è tutto il testo che il modello legge

1. Il **contesto** (*context*) è tutto il testo che il modello riceve ogni volta che risponde.
2. Oltre a ciò che ha imparato nell'addestramento, il modello conosce **solo ciò che è nel contesto**.
3. La **finestra di contesto** (*context window*) è la quantità massima di token che può leggere insieme: nei modelli più grandi arriva a circa **1 milione**.



Più il contesto è lungo, più il modello perde pezzi

1. I modelli a pagamento più avanzati (per esempio Claude Opus 5.5 e GPT-6.1-sol) lavorano in modo affidabile su contesti da **100.000 a 500.000 token**, a seconda del modello. 200.000 token sono più o meno *I promessi sposi*.
2. Le versioni **gratuite** leggono meno: ChatGPT e Gemini circa **30.000 token** (una quarantina di pagine); Claude fino a 200.000, ma è affidabile fino a circa **100.000**.
3. Un'informazione singola la ritrovano quasi sempre anche in testi più lunghi. Quando devono **collegare** più informazioni sparse, oltre questa soglia sbagliano più spesso.

COSA FARE

Per un argomento nuovo, apri una **chat nuova**.

Se la chat è lunga, chiedi un **riassunto** e incollalo in una chat nuova.



* La barra si riferisce ai modelli a pagamento.

File lunghi: l'app cerca i pezzi giusti (RAG)

1. Quando i documenti sono troppo grandi per il contesto, l'app **non li passa interi** al modello.
2. Prima **cerca** i pezzi più pertinenti alla domanda, poi mette nel contesto **solo quelli**. Si chiama *RAG*, *Retrieval-Augmented Generation*.
3. Se la ricerca sbaglia pezzo, il modello risponde «non c'è» oppure **inventa**, anche se l'informazione nel documento c'è.



> Con documenti lunghi indica **dove guardare**, per esempio: «Rispondi usando il capitolo 3».

Il modello conosce i fatti solo fino a una certa data

1. Il pre-addestramento usa testi raccolti fino a una certa data: la **data di aggiornamento delle conoscenze** (*knowledge cutoff*). Di solito è qualche mese prima dell'uscita del modello.
2. Di quello che è successo dopo il modello non sa niente, a meno che l'informazione non sia **nel contesto**.
3. La data di oggi la conosce perché l'app la scrive nel **prompt di sistema**; gli eventi recenti no.

PROVA

Disattiva la ricerca web e chiedi:

> «Senza cercare sul web: chi ha vinto le elezioni di metà mandato negli Stati Uniti del 3 novembre 2026?»

Poi riattiva la ricerca web e fai la stessa domanda.

→ **Cosa cambia tra le due risposte?**

Per i fatti recenti serve la **ricerca web**, che mette le informazioni nuove nel contesto.

Modelli di ragionamento: scrivono prima di rispondere

1. Un **modello di ragionamento** (*reasoning model*), prima della risposta, scrive un ragionamento passo per passo: prova, controlla, corregge.
2. Il ragionamento è fatto di token ed **entra nel contesto**: il modello lo rilegge mentre scrive la risposta, come una brutta copia.
3. Va molto meglio in matematica, logica e programmazione. È **più lento** e usa più token, quindi costa di più.

Ho 3 mele, ne mangio una, ne compro altre 5 e taglio a metà tutte le mele che ho. Due metà marciscono. Quante mezze mele posso ancora mangiare?

RAGIONAMENTO

```
Ho 3 mele, ne mangio una: 3 − 1 = 2.
Ne compro altre 5: 2 + 5 = 7.
Taglio tutto a metà: 7 × 2 = 14 metà.
Due marciscono: 14 − 2 = 12.
```

Risposta: 12 mezze mele.

Nelle app si attiva con pulsanti come **«Ragiona»** o **«Thinking»**. Serve per i problemi difficili, non per scrivere un'email.

Il modello scrive solo testo: gli strumenti eseguono

Uno **strumento** (*tool*) è un programma esterno che l'app mette a disposizione del modello. Il modello scrive una richiesta in un formato preciso, l'app la esegue e aggiunge il risultato al contesto.

1 **Il modello scrive una richiesta**

per esempio:
`cerca_web("meteo Trento")`

→

2 **L'app esegue lo strumento**

motore di ricerca, programma, mouse e tastiera

→

3 **Il risultato entra nel contesto**

come altro testo da leggere

→

4 **Il modello rilegge e continua**

risponde, oppure chiede un altro strumento

↺ si ripete finché il modello non scrive la risposta

L'elenco degli strumenti disponibili è scritto nel **prompt di sistema**: così il modello sa che esistono e come chiederli.

Ricerca web: il modello detta cosa cercare

utente

Che tempo farà domani a Trento?

modello

```
cerca_web("previsioni meteo Trento domani")
```

Testo scritto dal modello: di solito l'utente non lo vede

ricerca

1. ilmeteo.it · Trento domani: sole, minima 4 °C, massima 15 °C…
2. 3bmeteo.com · Meteo Trento: cielo sereno…

I risultati, aggiunti al contesto come testo

modello

Domani a Trento è previsto sole, con temperature tra 4 e 15 °C (fonte: ilmeteo.it).

La risposta, scritta leggendo i risultati

1. Il modello **scrive come testo la ricerca** da fare e poi **legge i risultati**, ancora una volta come testo.
2. La risposta dipende da **cosa ha cercato** e da **quali pagine ha trovato**: le fonti vanno controllate.

Codice: il modello lo scrive, un programma lo esegue

utente

Quante lettere "r" ci sono in «il ramarro marrone corre»?

modello

```
esegui_python(
  print("il ramarro marrone corre".count("r"))
)
```

Codice scritto dal modello: è testo come un altro

programma

```
7
```

Il programma esegue il codice e restituisce il risultato

modello

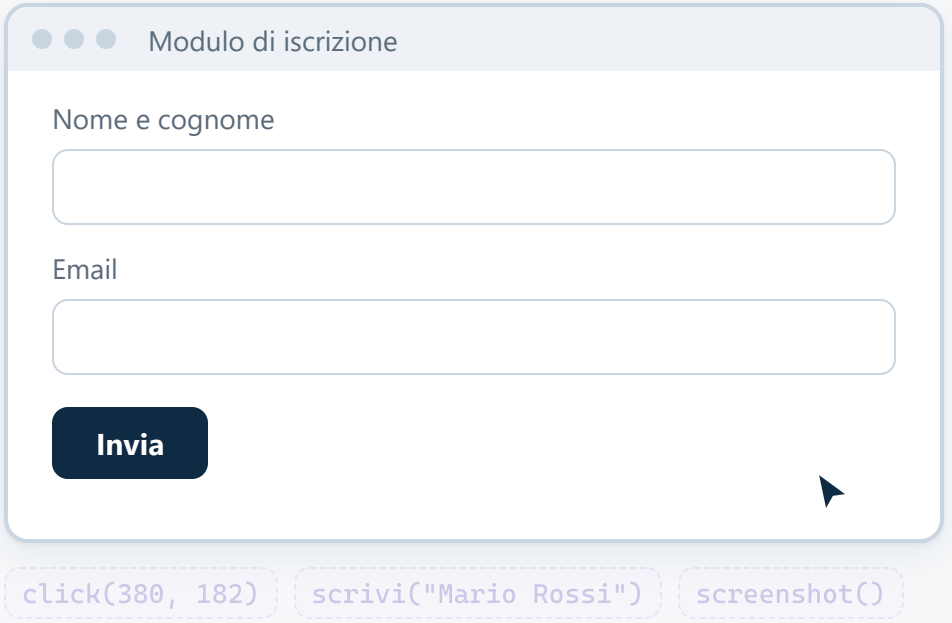
Nella frase ci sono 7 lettere «r».

La risposta, scritta leggendo il risultato

1. Il codice lo **scrive** il modello, ma a **eseguirlo** è un programma esterno.
2. Il numero è **calcolato dal programma**, non previsto token per token: per conteggi e calcoli è affidabile.

Uso del computer: il modello vede screenshot e scrive azioni

1. **Screenshot**: il modello riceve un'immagine dello schermo
2. **Azione scritta**: per esempio `click(380, 182)` o `scrivi("Mario Rossi")`
3. **Esecuzione**: un programma muove il mouse, clicca e digita
4. **Nuovo screenshot**: il modello guarda il risultato e decide l'azione successiva



> Il modello vede lo schermo facendo **uno screenshot alla volta**, e ogni azione è **un frammento di testo** (es. `cerca_web("meteo a Belluno")`).

Un agente lavora in ciclo finché il compito non è finito

Chat	Una domanda → una risposta. Il passo successivo lo decide **l'utente**.
Agente	Un obiettivo → molti passi. Il passo successivo lo decide **il modello**.

Un **agente** è un modello linguistico che usa gli strumenti molte volte di seguito, leggendo ogni volta il risultato e scegliendo da solo il passo successivo, finché il compito non è finito.



Non è un modello diverso: è lo **stesso modello linguistico**, messo in un ciclo e con degli strumenti a disposizione.

Un agente al lavoro, passo dopo passo

OBIETTIVO Confronta il prezzo della carta A4 di tre fornitori e prepara un file Excel.

	IL MODELLO SCRIVE	RISULTATO, AGGIUNTO AL CONTESTO
1	`cerca_web("carta A4 risma 500 fogli prezzo")`	Elenco di siti di fornitori
2	`leggi_pagina("fornitore-a.it/carta-a4")`	Risma da 500 fogli: 4,90 €
3	`leggi_pagina(…) per gli altri due fornitori`	Fornitore B: 4,20 € · Fornitore C: 5,10 €
4	`esegui_python( pd.DataFrame(prezzi).to_excel("prezzi.xlsx") )`	`NameError: name 'pd' is not defined`
5	`esegui_python( import pandas as pd` `pd.DataFrame(prezzi).to_excel("prezzi.xlsx") )`	File prezzi.xlsx creato
6	**«Ecco il file. Il fornitore più economico è B, con 4,20 € a risma.»**	

1 Nessuno gli ha detto quali passi fare: li ha scelti il modello, leggendo ogni volta il risultato precedente. Al passo 4 ha visto l'errore e l'ha **corretto da solo**.

2 Un errore in un passo può portarsi dietro anche i successivi. Se il compito comporta dei rischi, serve il **controllo di una persona**.

Modelli a diffusione: tutto il testo insieme

Autoregressivi
una parola alla volta, da sinistra a destra

La riunione di domani è spostata alle 15.

A diffusione
tutte le parole insieme, in pochi passaggi

La riunione di domani è spostata alle 15.

1. I modelli visti finora sono **autoregressivi**: scrivono un token alla volta e lo aggiungono al contesto.
2. Un modello **a diffusione** parte da una risposta fatta di segnaposto (*placeholder*) e in pochi passaggi riempie e corregge **tutte le posizioni insieme**.
3. Per questo è **molto più veloce**. Esempio: **Mercury** di Inception (chat.inceptionlabs.ai, senza account).
4. Lo svantaggio: per ora sono **meno capaci** dei migliori modelli autoregressivi.